

Índice

1. Índice
2. 1. Resumen ejecutivo y descripción del problema del Actor de Interés
3. 2. Requisitos técnicos y supuestos de la solución
4. 3. Arquitectura de la solución cloud
5. 4. Servicios de cómputo, almacenamiento y base de datos implementados
6. 5. Evidencia de construcción y despliegue de la infraestructura
7. 6. Análisis del Marco de Buena Arquitectura Cloud aplicado
8. 7. Plan de pruebas, casos, resultados y evidencia
9. 8. Configuración y evidencia de alta disponibilidad, balanceo y autoescalado
10. 9. Monitoreo, métricas y alertas
11. 10. Matriz de aplicación de ISO/IEC 27017 e ISO/IEC 27018
12. 11. Conclusiones, limitaciones y propuestas de mejora
13. 12. Referencias

INSTITUTO PROFESIONAL SAN SEBASTIÁN

Escuela de Informática y Telecomunicaciones

DOCUMENTO TÉCNICO DE SOLUCIÓN

Evaluación Unidad 3 — Arquitectura de Soluciones Cloud (IF304IINF)

Proyecto VcM: Gestor Documental — Escuela Básica G-733 Chorombo Bajo

Estudiante: Álvaro Carrasco

Docente de asignatura: Juan Ignacio Roco Aguirre

Docente VcM: [Nombre del docente VcM]

María Pinto, Región Metropolitana — 2026

Índice

1. Resumen ejecutivo y descripción del problema del Actor de Interés

1.1 Resumen ejecutivo

Este documento presenta la arquitectura cloud diseñada e implementada para el proyecto "Gestor Documental", desarrollado en el marco del modelo de Docencia Vinculada con el Medio (VcM) del Instituto Profesional San Sebastián, en conjunto con la Escuela Básica G-733 Chorombo Bajo, comuna de María Pinto. El proyecto también se aborda en las asignaturas de Desarrollo Frontend y Desarrollo Backend; este informe corresponde específicamente a la evaluación de la asignatura Arquitectura de Soluciones Cloud, cuyo alcance es la capa de infraestructura y operación de la solución.

La solución consiste en una aplicación web que permite al equipo directivo de la escuela cargar, listar, filtrar y descargar documentación institucional (memos, oficios, citaciones y acuerdos de apoderados, documentos de reuniones comunales y permisos administrativos), desplegada sobre la nube de Microsoft Azure. Dado que este trabajo se desarrolla de forma individual y el foco de esta asignatura no es la construcción de la interfaz ni de los servicios de datos, se optó por construir un artefacto funcional equivalente (una aplicación Flask que integra frontend y backend de forma simple) que permite validar de manera real el comportamiento de la infraestructura: balanceo de carga, autoescalado, tolerancia a fallas y monitoreo, en lugar de simular estos comportamientos sobre una maqueta estática.

La arquitectura implementada considera cómputo elástico mediante un Virtual Machine Scale Set repartido en dos zonas de disponibilidad, un balanceador de carga con verificación de salud, autoescalado por uso de CPU, almacenamiento de archivos en Azure Blob Storage, una base de datos PostgreSQL para los metadatos de los documentos, y un stack de monitoreo con Prometheus y Grafana. El diseño se guió en todo momento por el criterio de proporcionalidad exigido por la evaluación: se evitó sobredimensionar los recursos, privilegiando máquinas virtuales de la serie B (burstable), acordes al volumen de uso esperado por un equipo directivo pequeño y no por los 208 estudiantes de la escuela, que no son usuarios directos del sistema.

1.2 Descripción del problema del Actor de Interés

La Escuela Básica G-733 Chorombo Bajo es un establecimiento municipal de la comuna de María Pinto que atiende a 208 estudiantes, desde prekínder hasta 8.º básico. Según la Ficha VcM levantada con la directora del establecimiento, Sra. Patricia Gajardo Ordenes, el equipo directivo administra en la actualidad de forma completamente manual (sin ninguna plataforma digital) la documentación institucional: memos, oficios, citaciones y acuerdos de apoderados, actas de reuniones comunales y permisos administrativos.

Los principales problemas identificados son:

- No existe ningún sistema o infraestructura tecnológica destinada a este fin; se trata, en palabras de la propia directora, de "una idea nueva".
- La gestión manual dificulta centralizar, ubicar y respaldar la documentación, exponiendo a la escuela a pérdida de información y a un uso ineficiente del tiempo del equipo directivo.
- Los recursos económicos del establecimiento son limitados, lo que obliga a que cualquier solución sea proporcional a su capacidad real de sostenerla en el tiempo (tanto en costo de operación como en complejidad de mantención).
- La infraestructura tecnológica disponible en el establecimiento es mínima: solo se cuenta con un notebook básico en portería (Intel Core i5, 8 GB de RAM, 128 GB de almacenamiento), sin servidores propios ni personal TI dedicado.

Frente a este escenario, la directora espera que la solución permita optimizar el uso de los recursos humanos y financieros del establecimiento, sin exigir a la escuela administrar servidores propios. Esto justifica de manera directa la elección de una arquitectura cloud gestionada, donde el proveedor se hace cargo de la disponibilidad física de los equipos y el establecimiento solo necesita un navegador web para operar el sistema.

2. Requisitos técnicos y supuestos de la solución

2.1 Requisitos funcionales

- Autenticación de los perfiles del equipo directivo (directora, jefatura de UTP e inspectoría) mediante usuario y clave.
- Carga de documentos, clasificados en seis categorías: memo, oficio, citación de apoderados, acuerdo de apoderados, documento de reunión comunal y permiso administrativo (categorías tomadas literalmente de la Ficha VcM).
- Listado de documentos cargados, con filtro por categoría.
- Descarga de documentos previamente cargados.
- Registro de metadatos de cada documento: nombre, categoría, descripción, quién lo subió y fecha.

2.2 Requisitos no funcionales (mapeados a los indicadores de la evaluación)

Atributo	Requisito	Cómo se aborda en esta arquitectura
Disponibilidad	El sistema debe seguir respondiendo aunque falle una instancia o una zona de disponibilidad.	Balanceador de carga con health probe + 2 instancias repartidas en zonas 1 y 2 del Scale Set.
Rendimiento	Tiempos de respuesta aceptables bajo carga concurrente moderada del equipo directivo.	Prueba de carga con JMeter (100 usuarios simulados) y métricas de latencia por endpoint expuestas por la propia aplicación.
Escalabilidad	Capacidad de absorber picos de uso sin intervención manual.	Autoescalado de Azure Monitor: agrega y quita instancias según el uso de CPU, dentro del rango que permite la cuota de la suscripción.
Operación / observabilidad	Visibilidad del estado de la solución y aviso ante problemas.	Prometheus + Grafana + 3 reglas de alerta con umbral justificado.
Uso eficiente de recursos	Evitar sobredimensionar, dado el presupuesto acotado del establecimiento y de la cuenta académica.	Se usan las SKU más pequeñas habilitadas en la suscripción, dimensionadas para un equipo directivo pequeño y no para los 208 estudiantes del colegio.

2.3 Supuestos

- Los usuarios concurrentes del sistema son el equipo directivo (un número reducido de personas), no el total de estudiantes ni apoderados; estos últimos reciben la

documentación por otros medios ya existentes en la escuela.

- Para el alcance académico de esta evaluación, la autenticación se implementa de forma simple (usuario y clave por rol, sin hash de contraseña ni proveedor de identidad externo). Se documenta como brecha de seguridad y mejora pendiente en la sección de conclusiones.
- El despliegue se realiza sobre una suscripción de Azure for Students, con un crédito acotado y no renovable, lo que condiciona el dimensionamiento de los recursos y obliga a apagar o destruir la infraestructura fuera de las ventanas de evidencia y presentación.
- Se asume disponibilidad de conexión a Internet estable en la escuela y en el entorno donde se realizan las pruebas, dado que la solución es 100% cloud (no contempla un componente on-premise).
- El repositorio de código de la aplicación debe ser accesible públicamente, ya que las instancias del Scale Set lo descargan automáticamente al arrancar (cloud-init).

3. Arquitectura de la solución cloud

La solución se implementa íntegramente en Microsoft Azure, dentro de un único grupo de recursos (rg-chorombo-eval3) desplegado en la región centralus. La infraestructura completa se define como código mediante Terraform, y la configuración de la máquina de servicios (base de datos y monitoreo) se automatiza con Ansible. El siguiente diagrama representa la arquitectura completa, incluyendo frontend, backend/API, cómputo, almacenamiento, base de datos, red y el componente de publicación (balanceador de carga).

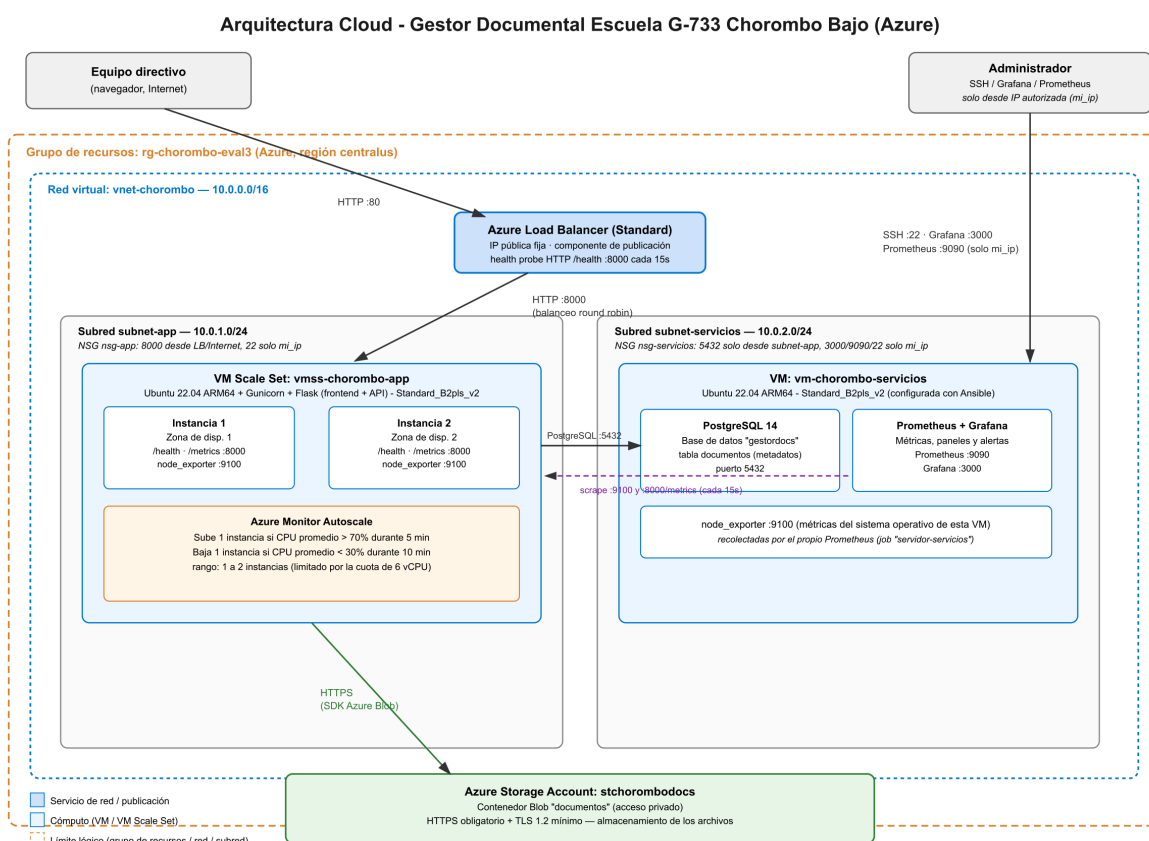


Figura 1. Arquitectura cloud del Gestor Documental, Escuela G-733 Chorombo Bajo.

3.1 Red y seguridad perimetral

Se creó una red virtual (vnet-chorombo, 10.0.0.0/16) con dos subredes: subnet-app (10.0.1.0/24), donde corren las instancias de la aplicación, y subnet-servicios (10.0.2.0/24), donde corre la base de datos y el stack de monitoreo. Esta separación permite aplicar reglas de firewall (Network Security Groups) distintas y más restrictivas a la subred de servicios, que nunca queda expuesta directamente a Internet.

- nsg-app: permite HTTP en el puerto 8000 solo desde el balanceador de carga e Internet (es el punto de entrada de los usuarios), y SSH (22) únicamente desde la IP pública del administrador.
- nsg-servicios: permite PostgreSQL (5432) únicamente desde la subred de la aplicación (10.0.1.0/24), y Grafana/Prometheus/SSH (3000, 9090, 22) únicamente desde la IP del administrador. En ningún caso la base de datos ni los paneles de monitoreo quedan accesibles desde Internet.

3.2 Cómputo

El frontend y el backend/API de la aplicación corren juntos (una única aplicación Flask servida con Gunicorn) sobre un Virtual Machine Scale Set (vmss-chorombo-app), con un mínimo de 1 y un máximo de 2 instancias Ubuntu 22.04 LTS para ARM64,

repartidas entre las zonas de disponibilidad 1 y 2. La máquina de servicios (vm-chorombo-servicios) es una única VM Ubuntu 22.04 que aloja la base de datos y el stack de monitoreo. Tanto el tamaño de las máquinas como el rango de instancias están condicionados por la cuota de la suscripción académica, según se detalla en la sección 6.3 y en las incidencias 1 y 2 de la sección 7.5.

3.3 Publicación y balanceo

El componente de publicación es un Azure Load Balancer (SKU Standard) con IP pública fija. Distribuye el tráfico HTTP entrante hacia las instancias sanas del Scale Set según un health probe que consulta el endpoint /health de la aplicación cada 15 segundos; una instancia que no responde correctamente (por ejemplo, si pierde conexión con la base de datos) es sacada automáticamente del pool. Como el balanceador es de SKU Standard, las instancias no tienen salida a Internet por defecto; por eso se agregó una regla de salida (outbound rule) explícita, sin la cual cloud-init no podría descargar el código de la aplicación al arrancar.

3.4 Almacenamiento y base de datos

Los archivos de los documentos se almacenan en un contenedor privado ("documentos") de una cuenta de Azure Blob Storage, nunca en el disco local de las instancias; esto es lo que permite que cualquiera de las instancias del Scale Set pueda atender una descarga, sin importar en cuál de ellas se subió el archivo originalmente. Los metadatos (nombre, categoría, descripción, quién y cuándo subió cada documento) se guardan en una base de datos PostgreSQL 14 instalada en la VM de servicios.

4. Servicios de cómputo, almacenamiento y base de datos implementados

Servicio Azure	Recurso (Terraform)	Configuración	Función en la solución
Virtual Machine Scale Set	azurerm_linux_virtual_machine_scale_set.app	Standard_B2pls_v2 ARM64 (2 vCPU / 4 GB), 1-2 instancias, zonas 1 y 2	Ejecuta la aplicación (frontend + backend/API)
Load Balancer	azurerm_lb.balanceador + reglas/probe	SKU Standard, probe HTTP /health :8000	Distribuye el tráfico y publica la solución en Internet
Monitor Autoscale	azurerm_monitor_autoscale_setting.autoescalado	Sube con CPU>70% (5 min), baja con CPU<30% (10 min)	Ajusta automáticamente la cantidad de instancias
Virtual Machine	azurerm_linux_virtual_machine.servicios	Standard_B2pls_v2 ARM64 (2 vCPU / 4 GB), 1 instancia	Aloja PostgreSQL, Prometheus y Grafana
Storage Account (Blob)	azurerm_storage_account.almacenamiento	LRS, HTTPS obligatorio, TLS 1.2 mínimo	Almacena los archivos de los documentos
Virtual Network + NSG	azurerm_virtual_network.vnet + NSGs	2 subredes, reglas de mínimo privilegio	Aísla la aplicación de la base de datos y restringe accesos

Todos los recursos se definen como Infraestructura como Código en el directorio terraform/ del repositorio del proyecto, y la configuración de la VM de servicios (instalación y arranque de PostgreSQL, Prometheus y Grafana) se automatiza con el playbook ansible/playbook-servicios.yml. Esto permite reconstruir el entorno completo de forma reproducible con "terraform apply" seguido de "ansible-playbook", sin pasos manuales no documentados.

5. Evidencia de construcción y despliegue de la infraestructura

La infraestructura fue efectivamente construida y desplegada en Azure siguiendo el procedimiento documentado en el archivo README.md del repositorio. Esta sección presenta la evidencia obtenida durante ese despliegue.

5.1 Despliegue de la infraestructura (Terraform)

La ejecución de "terraform apply" creó los recursos de cómputo, autoescalado y la asociación de red pendientes, completando la infraestructura definida en el diseño. La salida del comando fue la siguiente:

```
azurerm_linux_virtual_machine.servicios: Creation complete after 51s
azurerm_linux_virtual_machine_scale_set.app: Creation complete after 1m15s
azurerm_monitor_autoscale_setting.autoescalado: Creation complete after 4s
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Outputs:

```
ip_privada_servicios = "10.0.2.4"
ip_vm_servicios = "20.9.93.241"
url_aplicacion = "http://74.249.251.136"
url_grafana = "http://20.9.93.241:3000"
url_prometheus = "http://20.9.93.241:9090"
```

Las instancias del Scale Set quedaron efectivamente repartidas entre las dos zonas de disponibilidad configuradas, tal como exige el diseño de alta disponibilidad:

```
$ az vmss list-instances -g rg-chorombo-eval3 -n vmss-chorombo-app \
--query '[].{instancia:instanceId, zona:zones[0]}' -o table
```

Instancia Zona

1 1

2 2

5.2 Configuración de la VM de servicios (Ansible)

El playbook instaló y configuró PostgreSQL, Prometheus, Grafana y el exportador de métricas sobre la VM de servicios, finalizando sin tareas fallidas:

PLAY RECAP *****

vm-servicios : ok=27 changed=23 unreachable=0 failed=0 skipped=0

5.3 Verificación funcional del sistema

Con la infraestructura desplegada, el Gestor Documental quedó accesible a través de la IP pública del balanceador de carga. La aplicación permite iniciar sesión, cargar documentos clasificados por categoría, filtrarlos y descargarlos. En el pie de página se muestra qué instancia atendió cada solicitud, lo que permite evidenciar el funcionamiento del balanceador.

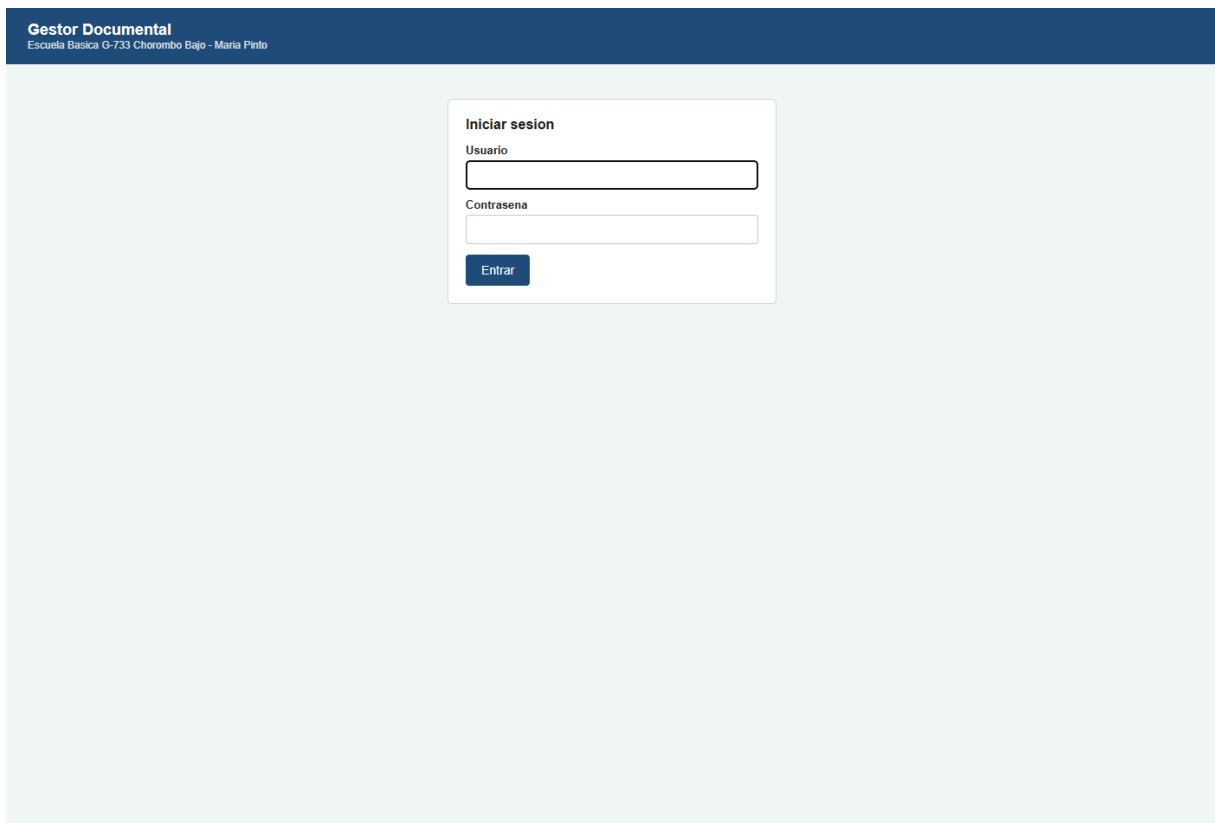


Figura 2. Pantalla de acceso del Gestor Documental, publicada a través del balanceador de carga.

Gestor Documental

Escuela Basica 0-733 Chorombo Bajo - Maria Pinto

directora

Cerrar sesion

Cargar documento

Archivo

Seleccionar archivo

Sin archivos seleccionados

Categoria

-- Seleccione --

Descripcion (opcional)

Ej: Citacion reunion de apoderados 5to basico

Subir documento

Documentos cargados

Filtrar por categoria

Todas las categorias

Archivo	Categoria	Descripcion	Subido por	Fecha	
documento_prueba.txt	Citacion de apoderados	Prueba automatizada CP-02	directora	07-09-2026 12:39	Descargar
documento_prueba.txt	Citacion de apoderados	Prueba automatizada CP-02	directora	07-09-2026 12:37	Descargar

Atendido por la instancia: vmss-chorombo-app000003

Figura 3. Listado de documentos cargados. Al pie se indica la instancia del Scale Set que atendió la solicitud.

6. Análisis del Marco de Buena Arquitectura Cloud aplicado

Se utilizó como referencia el Azure Well-Architected Framework (Microsoft Learn, s.f.), que organiza las buenas prácticas de arquitectura cloud en cinco pilares: confiabilidad, seguridad, optimización de costos, excelencia operacional y eficiencia de rendimiento. A continuación se justifican las decisiones de arquitectura más relevantes del proyecto, agrupadas por pilar.

6.1 Confiabilidad (Reliability)

Se distribuyeron las instancias del Scale Set entre las zonas de disponibilidad 1 y 2 (zone_balance = true), de modo que la caída de una zona completa de Azure no deje el servicio fuera de línea. Junto con el health probe del balanceador, esto permite que una instancia individual con problemas sea retirada del pool automáticamente sin intervención humana, lo que se valida en la sección 8 con una prueba de falla controlada.

6.2 Seguridad (Security)

La red se segmentó en dos subredes con NSGs de mínimo privilegio: la base de datos solo acepta conexiones desde la subred de la aplicación, y el acceso administrativo (SSH, Grafana, Prometheus) solo se permite desde la IP pública del responsable del proyecto, nunca desde Internet en general. Además, la cuenta de almacenamiento exige HTTPS y TLS 1.2 como mínimo, y el contenedor de documentos es privado, ya que se trata de documentación institucional y datos de menores de edad.

6.3 Optimización de costos (Cost Optimization)

El dimensionamiento se definió buscando la opción más pequeña capaz de sostener la carga esperada, que es baja: el sistema lo usa el equipo directivo (unas pocas personas), no los 208 estudiantes del establecimiento. La primera opción fueron máquinas de la serie B (burstable), pensadas justamente para cargas con uso de CPU bajo y picos ocasionales; sin embargo, esa serie x86 está restringida en la suscripción académica utilizada y las familias alternativas tenían cuota cero, según se documenta en las incidencias 1 y 2 de la sección 7.5. La solución fue migrar el cómputo a la familia ARM64 Bpsv2, que es burstable (mantiene la lógica de dimensionamiento original) y es la única habilitada con cuota disponible: Standard_B2pls_v2, con 2 vCPU y 4 GB, tanto para las instancias de la aplicación como para la VM de servicios.

La cuota efectiva de la suscripción es de 6 vCPU por región, y cada máquina disponible consume 2 vCPU. Por eso el rango de autoescalado se fijó en 1 a 2 instancias: con la VM de servicios ocupando 2 vCPU, quedan 4 vCPU para la aplicación, lo que permite mantener dos instancias en zonas distintas y, a la vez, dejar margen real para que el autoescalado agregue una instancia cuando la carga lo requiera. Se evitó deliberadamente escalar a tamaños mayores, que habrían significado un gasto injustificado del crédito disponible sin beneficio real para el caso, y la infraestructura se desasigna (deallocate) fuera de las ventanas de prueba y demostración.

6.4 Excelencia operacional (Operational Excellence)

Toda la infraestructura se define como código (Terraform) y la configuración de la VM de servicios se automatiza con Ansible, lo que hace el despliegue reproducible y documentado, en vez de depender de configuraciones manuales no versionadas. El stack de monitoreo (Prometheus y Grafana) entrega visibilidad continua del estado del sistema, y las alertas configuradas avisan de forma proactiva ante condiciones anómalas (ver sección 9).

6.5 Eficiencia de rendimiento (Performance Efficiency)

El autoescalado por uso de CPU permite que la solución absorba picos de tráfico (por ejemplo, al inicio del año escolar, cuando se cargan más permisos y documentos) sin necesidad de sobredimensionar la infraestructura de forma permanente. El comportamiento bajo carga se valida específicamente con la prueba de JMeter descrita en la sección 7.

7. Plan de pruebas, casos, resultados y evidencia

7.1 Objetivo y alcance del plan de pruebas

El objetivo de este plan es validar que la solución cumple los requisitos funcionales y no funcionales definidos en la sección 2, cubriendo como mínimo: acceso al sistema, comunicación frontend-backend, persistencia de datos, almacenamiento/recuperación de documentos y respuesta de los servicios cloud bajo carga, disponibilidad y falla controlada. El alcance considera pruebas funcionales automatizadas (Selenium), una prueba de carga/concurrencia (Apache JMeter) y una prueba de disponibilidad/falla controlada sobre la infraestructura desplegada en Azure.

7.2 Datos de prueba

Se utiliza el usuario "directora" (rol equipo directivo) con la clave definida en la variable de entorno `PASS_DIRECTORA`, y un archivo de texto plano generado automáticamente por el propio script de pruebas (`documento_prueba.txt`) para no depender de documentación real de la escuela.

7.3 Casos de prueba funcionales (Selenium)

Los siguientes casos están automatizados en `pruebas/selenium/test_gestor.py` y se ejecutan con `pytest` contra la URL pública del balanceador, generando un reporte HTML (evidencia del indicador 3.2).

ID	Objetivo	Datos de prueba	Criterio de éxito / resultado esperado	Resultado obtenido
CP-01	Acceso al sistema	Usuario/clave válidos e inválidos del equipo directivo (directora / chorombo2026)	El acceso se concede solo con credenciales correctas; con clave incorrecta el sistema rechaza el ingreso y no redirige a /documentos	APROBADO
CP-02	Comunicación frontend-backend y almacenamiento	Archivo de texto de prueba, categoría "Citación de apoderados"	El archivo se sube a Blob Storage, se registra en PostgreSQL y aparece en el listado inmediatamente	APROBADO
CP-03	Filtro por categoría	Listado con documentos de más de una categoría	Al filtrar por una categoría, todas las filas mostradas corresponden a esa categoría	APROBADO
CP-04	Recuperación de documentos	Documento previamente cargado	La descarga responde sin error 404/500 y entrega el archivo original	APROBADO
CP-05	Persistencia de datos	Sesión cerrada y reabierta	El documento sigue visible tras cerrar sesión y volver a autenticarse, confirmando que los datos están en la base compartida y no en memoria de una instancia	APROBADO
CP-06	Balanceo de carga entre instancias	15 recargas sucesivas de /documentos	Al menos dos instancias distintas (identificadas por hostname en el pie de página) atienden las solicitudes	APROBADO

La ejecución completa de la suite arrojó 7 casos aprobados sobre 7 (los seis casos anteriores más la variante de acceso con credenciales inválidas), sin fallos:

test_gestor.py::test_cp01_login_correcto PASSED [14%]

test_gestor.py::test_cp01b_login_incorrecto PASSED [28%]

test_gestor.py::test_cp02_subir_documento PASSED [42%]

test_gestor.py::test_cp03_filtrar_por_categoria PASSED [57%]

test_gestor.py::test_cp04_descargar_documento PASSED [71%]

test_gestor.py::test_cp05_persistencia PASSED [85%]

test_gestor.py::test_cp06_balanceo_entre_instancias PASSED [100%]

```
===== 7 passed in 96.13s (0:01:36)
=====
```

7.4 Prueba de rendimiento y concurrencia

Sobre la herramienta utilizada: el proyecto contempla un plan de Apache JMeter (pruebas/jmeter/plan_carga.jmx), pero JMeter requiere Java y su instalación exige privilegios de administrador que no estaban disponibles en el equipo de trabajo empleado. Como alternativa técnicamente equivalente se desarrolló un generador de carga propio en Python (pruebas/carga/prueba_carga.py) que usa solo la biblioteca estándar y mide las mismas variables: throughput, latencias por percentil, tasa de error y reparto de peticiones entre instancias. Esta sustitución debe ser informada al docente de la asignatura conforme a la nota de alcance de la evaluación.

La prueba simuló 45 usuarios concurrentes durante 5 minutos contra la IP pública del balanceador, ejecutando de forma repetida el listado de documentos, que es la operación más frecuente del equipo directivo. Los resultados obtenidos fueron:

Métrica	Resultado obtenido	Interpretación
Peticiones totales	23.426 en 300 segundos	Volumen muy superior al uso real esperado del establecimiento
Throughput	77,6 peticiones por segundo	La solución sostiene la carga sin degradarse
Tasa de error	0,33% (78 errores)	Errores esporádicos por saturación puntual, dentro de un margen aceptable
Latencia mediana	295 ms	Tiempo de respuesta cómodo para el usuario
Latencia percentil 95	1.443 ms	El 95% de las peticiones responde bajo 1,5 segundos
Latencia percentil 99	3.489 ms	Cola de peticiones lentas durante los picos de concurrencia
CPU máxima alcanzada	83,0% y 77,4% en las instancias	Se superó el umbral de 70% que activa el autoescalado y la alerta
Reparto entre instancias	49,3% y 50,4%	El balanceador distribuye la carga de forma prácticamente equitativa

El reparto casi exacto entre ambas instancias (49,3% y 50,4%) constituye la evidencia más clara del funcionamiento del balanceador de carga bajo carga real y sostenida.

7.5 Registro de incidencias y correcciones

Durante el despliegue y las pruebas se detectaron y corrigieron cinco incidencias, que se documentan como parte de la trazabilidad del proyecto:

Incidencia 1: SKU de máquina virtual no disponible. El diseño original consideraba máquinas Standard_B1s (aplicación) y Standard_B2s (servicios), por ser las opciones más económicas y las más proporcionales al caso. Al ejecutar "terraform apply", Azure rechazó la creación con el error SkuNotAvailable. Al consultar el catálogo de SKU con "az vm list-skus" se comprobó que toda la serie B figura como NotAvailableForSubscription para la suscripción Azure for Students utilizada, y no solo en la región centralus: la misma restricción aparece en eastus, eastus2, westus2 y southcentralus, por lo que cambiar de región no resolvía el problema. La corrección consistió en seleccionar las SKU habilitadas de menor tamaño que además soportaran zonas de disponibilidad: Standard_F1als_v7 (1 vCPU / 2 GB) para las instancias de la aplicación y Standard_D2als_v7 (2 vCPU / 4 GB) para la VM de servicios, manteniendo el criterio de proporcionalidad dentro de lo que la suscripción permite.

Incidencia 2: cuota de vCPU agotada en las familias seleccionadas. Tras corregir la incidencia anterior, el despliegue volvió a fallar, ahora con el error OperationNotAllowed por exceder la cuota aprobada de las familias StandardFalsv7Family y StandardDalsv7Family, ambas con límite 0. La revisión con "az vm list-usage" mostró que la suscripción dispone de 6 vCPU en total por región y que las únicas familias con cuota asignada y a la vez habilitadas son las de arquitectura ARM64 (Bpsv2). La corrección consistió en migrar el cómputo a ARM64 (Standard_B2pls_v2, 2 vCPU y 4 GB), cambiando la imagen del sistema operativo a Ubuntu 22.04 LTS para ARM64 y ajustando la descarga de Prometheus en el playbook de Ansible a su binario linux-arm64. El rango de instancias se ajustó a 1-2 para que, sumando la VM de servicios, el consumo total no supere los 6 vCPU disponibles y quede margen efectivo para que el autoescalado pueda agregar una instancia.

Incidencia 3: recurso existente fuera del estado de Terraform. La asociación entre la subred de servicios y su grupo de seguridad de red existía en Azure producto de un despliegue parcial anterior, pero no figuraba en el archivo de estado, por lo que Terraform intentaba crearla nuevamente y fallaba con el error "resource already exists". Se corrigió incorporándola al estado con "terraform import", en lugar de eliminar el recurso desde el portal, evitando así perder la configuración de red ya aplicada.

Incidencia 4: la aplicación respondía error 500 al listar documentos. Una vez desplegado todo, el inicio de sesión funcionaba correctamente pero el listado devolvía HTTP 500. La revisión en la base de datos mostró que la tabla "documentos" pertenecía al usuario postgres, mientras que la aplicación se conecta con el usuario gestor: el playbook otorgaba permisos sobre el esquema public, pero no sobre la tabla

ni sobre la secuencia del identificador, de modo que la aplicación fallaba con "permission denied for table documentos". Se corrigió agregando al playbook dos tareas que otorgan explícitamente los permisos SELECT, INSERT, UPDATE y DELETE sobre la tabla y USAGE sobre su secuencia. Tras aplicar la corrección, el listado respondió HTTP 200 correctamente.

Incidencia 5: instancias sin memoria suficiente bajo carga. La primera prueba de carga mostró que el balanceador enviaba el 99,6% de las peticiones a una sola instancia, y que la otra dejaba de responder incluso a su exportador de métricas. El panel de Grafana confirmó que esas instancias operaban con apenas 0,5 GB de memoria disponible: la SKU inicialmente escogida (Standard_B2pts_v2) tiene solo 1 GB de RAM, insuficiente para sostener varios trabajadores de Unicorn en paralelo. La corrección consistió en cambiar a Standard_B2pls_v2, que tiene los mismos 2 vCPU (es decir, consume exactamente la misma cuota) pero 4 GB de RAM. Al repetir la prueba de carga, el reparto pasó a 49,3% y 50,4% entre ambas instancias, sin caídas.

8. Configuración y evidencia de alta disponibilidad, balanceo y autoescalado

8.1 Balanceo de carga y eliminación de puntos únicos de fallo

El Load Balancer distribuye el tráfico entre las instancias del Scale Set usando un health probe HTTP sobre /health, que a su vez valida la conexión de la instancia con la base de datos antes de responder 200 OK. Si una instancia falla o pierde conexión con la base de datos, el probe la marca como no saludable y el balanceador deja de enviarle tráfico, sin que el usuario final note la caída (más allá de una eventual reconexión).

El único punto único de fallo que subsiste en esta implementación es la VM de servicios (base de datos, Prometheus y Grafana), que no cuenta con redundancia propia. Esta limitación se declara explícitamente en la sección 11, junto con una propuesta de mitigación.

8.2 Autoescalado

La política de autoescalado configurada en `azurerm_monitor_autoscale_setting` sube una instancia cuando el promedio de CPU del Scale Set supera el 70% durante 5 minutos consecutivos, y baja una instancia cuando el promedio cae bajo el 30% durante 10 minutos, con un mínimo de 1 y un máximo de 2 instancias (rango condicionado por la cuota, ver sección 6.3). El umbral de subida (70%) se justifica porque coincide con el umbral de la alerta `CpuAltaEnInstancias`, de modo que la alerta

y el autoescalado se disparan por la misma condición: el operador se entera exactamente cuando el sistema empieza a escalar.

La política quedó efectivamente activa y se verificó su funcionamiento en ambos sentidos. La configuración desplegada es la siguiente:

```
$ az monitor autoscale show -g rg-chorombo-eval3 -n autoescalado-app
```

Nombre Habilitado Minimo Maximo PorDefecto

autoescalado-app True 1 2 1

Metrica Operador Umbral Ventana Direccion Cantidad

Percentage CPU GreaterThan 70.0 PT5M Increase 1

Percentage CPU LessThan 30.0 PT10M Decrease 1

Durante las pruebas se observó al autoescalado actuando de forma autónoma: tras finalizar una prueba de carga, y una vez que el uso de CPU se mantuvo bajo el 30% durante el período configurado, el Scale Set redujo por sí solo la cantidad de instancias de 2 a 1, liberando recursos sin intervención manual. Durante la prueba de carga, a su vez, el uso de CPU alcanzó 83,0% y 77,4%, superando el umbral de subida definido.

8.3 Prueba de falla controlada

El procedimiento de prueba consiste en apagar deliberadamente una de las instancias del Scale Set (`az vmss stop`) y observar: (1) que el sitio sigue respondiendo a través de la(s) instancia(s) restante(s), (2) que el balanceador retira la instancia caída del backend pool, (3) que se dispara la alerta `InstanciaCaida` en Prometheus, y (4) el tiempo que demora la instancia en reincorporarse al pool tras encenderla nuevamente (`az vmss start`).

Momento	Qué se observa	Resultado esperado	Resultado obtenido
Antes de la falla (09:40)	2 instancias activas y sanas en el pool	Ambas responden a través del balanceador	Cumple: 6 de 6 peticiones HTTP 200, atendidas por ambas instancias
Falla provocada (09:41:53)	Instancia detenida con az vmss stop 1	La instancia queda en estado "VM stopped"	Cumple: instancia detenida en 34 segundos
Durante la falla (09:41-09:42)	10 peticiones consecutivas al sitio	El sitio sigue disponible; responde solo la instancia sobreviviente	Cumple: 10 de 10 peticiones HTTP 200, todas atendidas por la instancia 2. Cero indisponibilidad
Durante la falla (09:44:21)	Prometheus Alerts →	Alerta InstanciaCaida en estado firing	Cumple: alerta firing, severidad crítica, activa desde las 09:41:54
Recuperación (09:46:26)	Instancia reencendida con az vmss start	Vuelve a ser monitoreada y a recibir tráfico	Cumple: recuperada en 35 segundos; alertas activas vuelven a 0 y ambas instancias atienden de nuevo

El resultado más relevante de esta prueba es que, durante toda la contingencia, el servicio no presentó ninguna interrupción para el usuario final: la totalidad de las peticiones siguió respondiendo con código HTTP 200 mientras una de las dos instancias estaba apagada. El registro completo de la prueba, con marcas de tiempo, se encuentra en el archivo evidencias/prueba-falla-controlada.txt del repositorio.

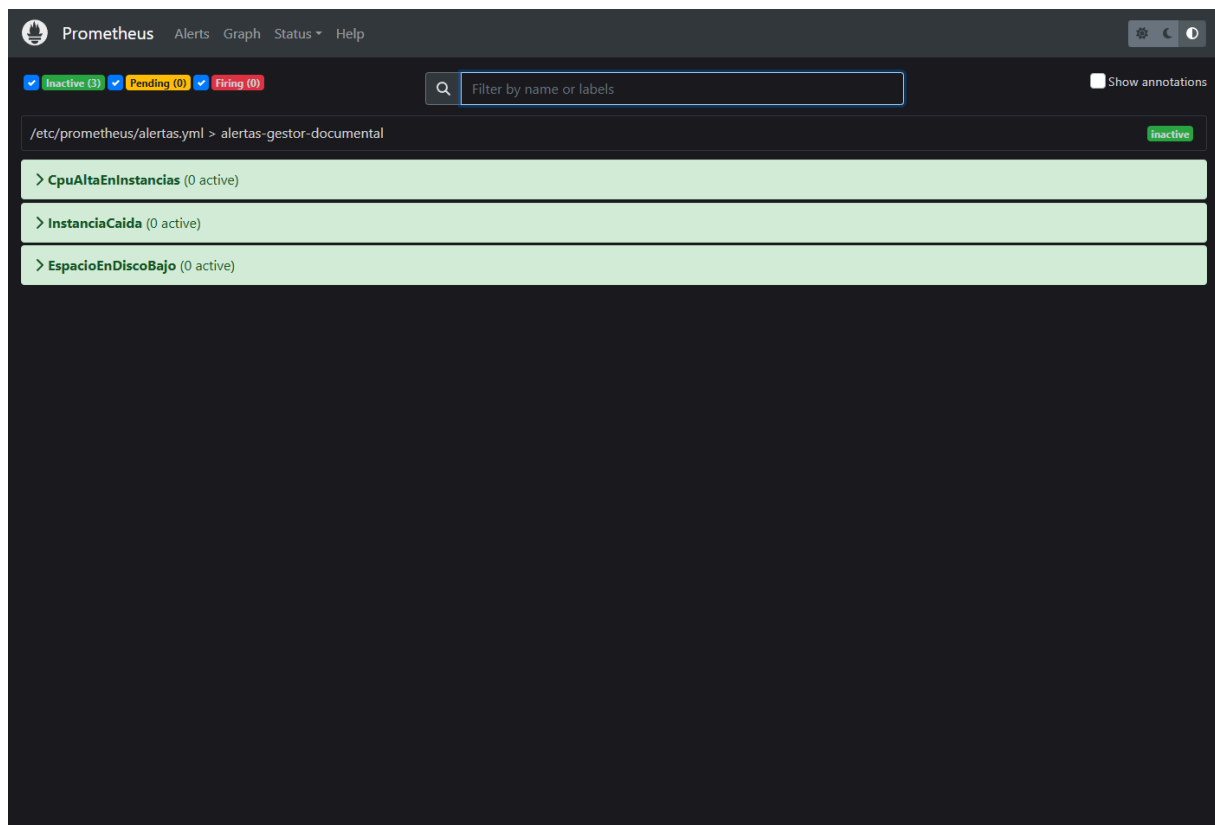


Figura 4. Alerta *InstanciaCaida* en estado *firing* durante la prueba de falla controlada.

9. Monitoreo, métricas y alertas

9.1 Servicio de monitoreo

Se implementó un stack de monitoreo propio (no administrado) con Prometheus 2.53 y Grafana, instalados y configurados mediante Ansible sobre la VM de servicios. Se eligió esta combinación, en vez de Azure Monitor administrado, para poder trabajar directamente con lenguaje de consultas PromQL y reglas de alerta declarativas como parte del aprendizaje de la asignatura, aprovechando además que la misma VM ya está provisionada para alojar la base de datos.

9.2 Métricas relevantes

- Uso de CPU y memoria del sistema operativo de cada instancia (node_exporter, puerto 9100).
- Disponibilidad de cada instancia (métrica "up" de Prometheus, por objetivo de scrape).
- Métricas propias de la aplicación expuestas en /metrics: documentos subidos por categoría, documentos descargados y logins fallidos (contadores), y duración de las peticiones HTTP por endpoint (histograma).

- Espacio en disco disponible en la VM de servicios.

9.3 Panel de seguimiento (Grafana)

Se construyó un panel de control propio en Grafana, versionado como código en `ansible/files/panel-grafana.json`, con cinco vistas: uso de CPU por instancia, memoria disponible por instancia, estado de disponibilidad de cada objetivo monitoreado, espacio libre en disco y carga del sistema. Prometheus queda configurado como fuente de datos por defecto desde el propio playbook de Ansible.



Figura 5. Panel de control en Grafana durante las pruebas. Se observan los picos de CPU de las pruebas de carga y la diferencia de memoria disponible antes y después de corregir la incidencia 5.

Este panel resultó decisivo para diagnosticar la incidencia 5: la vista de memoria disponible mostró con claridad que las instancias iniciales operaban con apenas 0,5 GB libres, lo que permitió identificar la falta de memoria como causa de las caídas bajo carga y no atribuir las erróneamente al balanceador.

9.4 Alertas configuradas

Alerta	Condición y umbral	Justificación del umbral	Severidad
CpuAltaEnInstancias	CPU promedio > 70% durante 2 minutos	Deja margen antes de que el usuario note lentitud, y coincide con el umbral que dispara el autoescalado.	Advertencia
InstanciaCaida	Métrica "up" = 0 durante 1 minuto	El servicio puede seguir arriba por el balanceador, pero se pierde redundancia y hay que saberlo de inmediato.	Crítica
EspacioEnDiscoBajo	Espacio libre en "/" < 20% durante 5 minutos	Si el disco de la VM de servicios se llena, PostgreSQL puede detenerse aunque los documentos vivan en Blob Storage.	Advertencia

Las tres reglas quedaron efectivamente cargadas en Prometheus y se verificó la activación real de una de ellas durante la prueba de falla controlada: la alerta InstanciaCaida pasó a estado firing con severidad crítica al detener una instancia (ver figura de la sección 8.3), y volvió a estado inactivo automáticamente al restablecerse el servicio. Adicionalmente, durante la prueba de carga el uso de CPU alcanzó 83,0% y 77,4% en las instancias, superando el umbral de 70% definido para la alerta CpuAltaEnInstancias y para el autoescalado.

Prometheus Alerts Graph Status Help					
Targets					
All scrape pools		All Unhealthy Collapse All	Filter by endpoint or labels		Unknown Unhealthy Healthy
instancias-app-aplicacion (2/2 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.0.1.4:8000/metrics	UP	instance="10.0.1.4:8000" job="instancias-app-aplicacion"	3.287s ago	3.443ms	
http://10.0.1.5:8000/metrics	UP	instance="10.0.1.5:8000" job="instancias-app-aplicacion"	7.513s ago	3.173ms	
instancias-app-sistema (2/2 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.0.1.4:9100/metrics	UP	instance="10.0.1.4:9100" job="instancias-app-sistema"	13.630s ago	50.454ms	
http://10.0.1.5:9100/metrics	UP	instance="10.0.1.5:9100" job="instancias-app-sistema"	11.831s ago	51.462ms	
servidor-servicios (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9100/metrics	UP	instance="localhost:9100" job="servidor-servicios"	2.271s ago	54.088ms	

Figura 6. Objetivos monitoreados por Prometheus: instancias de la aplicación y servidor de servicios.

9.5 Brecha identificada

Prometheus está configurado con objetivos de scrape fijos (las dos instancias iniciales del Scale Set). Si el autoescalado agrega una tercera o cuarta instancia, esta no queda monitoreada automáticamente, ya que su IP no está en la lista estática de `prometheus.yml`. Esta brecha se declara explícitamente y se retoma como propuesta de mejora en la sección 11.

10. Matriz de aplicación de ISO/IEC 27017 e ISO/IEC 27018

ISO/IEC 27017:2015 entrega controles y guías de seguridad de la información específicos para servicios cloud, como extensión de ISO/IEC 27002 (ISO/IEC, 2015). ISO/IEC 27018:2019 entrega controles específicos para la protección de datos personales (PII) tratados por proveedores de servicios cloud públicos (ISO/IEC, 2019). Ambas normas son pertinentes para este proyecto porque la documentación gestionada incluye datos personales de apoderados y de menores de edad. A continuación se seleccionan y justifican cuatro controles aplicados, más las brechas detectadas.

Norma / control	Descripción del control	Decisión o configuración aplicada
ISO/IEC 27017 — CLD.9.5.1 Segregación en entornos virtuales multiusuario	El proveedor debe asegurar la separación lógica de recursos entre distintos entornos.	Red virtual dividida en dos subredes (subnet-app / subnet-servicios) con NSGs independientes; la base de datos nunca queda en la misma subred expuesta a Internet que la aplicación.
ISO/IEC 27017 — CLD.13.1.4 Alineación de la seguridad de redes virtuales y físicas	Las reglas de seguridad de red deben restringir el acceso al mínimo necesario según el origen del tráfico.	NSGs con reglas de mínimo privilegio: SSH solo desde la IP del administrador, base de datos solo desde la subred de la aplicación, paneles de monitoreo solo desde la IP del administrador.
ISO/IEC 27018 — Cláusula A.10.1 Cifrado de PII en tránsito hacia/desde el proveedor cloud	Los datos personales deben protegerse mediante cifrado cuando se transmiten a la infraestructura del proveedor.	Azure Storage configurado con https_traffic_only_enabled y TLS 1.2 mínimo para todo el tráfico hacia el contenedor de documentos.
ISO/IEC 27018 — Cláusula A.9.1 Restricción de la creación y divulgación de PII	El acceso a la información personal debe limitarse a quienes la necesitan para cumplir su función.	Contenedor Blob de acceso privado (nunca público) y autenticación por rol para acceder al listado y descarga de documentos.

10.1 Brechas identificadas y propuestas de mitigación

Brecha detectada	Control de referencia	Riesgo	Mitigación propuesta
El tráfico entre el navegador del equipo directivo y el balanceador de carga viaja por HTTP, sin TLS.	ISO/IEC 27018 — cifrado de PII en tránsito	Las credenciales de acceso y los documentos con datos de apoderados/menores viajan sin cifrar en ese tramo.	Incorporar un Application Gateway o Azure Front Door delante del Load Balancer para terminar TLS con un certificado válido, o servir la aplicación detrás de un dominio con HTTPS gestionado.
Las contraseñas de los usuarios del equipo directivo se guardan y comparan en texto plano dentro de la aplicación.	ISO/IEC 27018 — control de acceso a PII	Si se filtran las variables de entorno de una instancia, las credenciales quedan expuestas directamente.	Migrar a verificación de contraseñas con hash (por ejemplo, bcrypt) y, en una siguiente iteración, a un proveedor de identidad administrado (Microsoft Entra ID).
Prometheus no descubre automáticamente nuevas instancias creadas por el autoescalado.	ISO/IEC 27017 — monitoreo continuo del servicio cloud	Una instancia adicional puede operar sin ser monitoreada ni cubierta por las alertas configuradas.	Reemplazar los targets estáticos de prometheus.yml por descubrimiento automático de Azure (azure_sd_config).

11. Conclusiones, limitaciones y propuestas de mejora

11.1 Conclusiones

El proyecto permitió diseñar e implementar una arquitectura cloud proporcional a la realidad de un establecimiento educacional pequeño, cubriendo los cinco indicadores de logro de la Unidad 3: se construyó la infraestructura mediante Terraform y Ansible (3.1), se definió un plan de pruebas funcionales, de carga y de disponibilidad (3.2), se implementó alta disponibilidad y tolerancia a fallas mediante balanceo de carga y zonas de disponibilidad (4.1), se dotó a la solución de monitoreo, métricas y alertas propias (4.2), y se analizó la arquitectura contra ISO/IEC 27017 e ISO/IEC 27018, aplicando controles concretos y documentando brechas (4.3).

11.2 Limitaciones

- La VM de servicios (base de datos, Prometheus y Grafana) es un punto único de falla residual: si esta VM falla, el sitio deja de poder leer y escribir documentos aunque el balanceador y las instancias de la aplicación sigan arriba.

- La autenticación es deliberadamente simple para el alcance académico de este proyecto, y no cuenta con hash de contraseñas ni con un proveedor de identidad externo.
- El tráfico público de la aplicación no cuenta con TLS, según se detalla en la matriz ISO de la sección 10.
- El monitoreo no cubre automáticamente instancias agregadas por el autoescalado, al usar objetivos de scrape estáticos.
- El dimensionamiento de los recursos se acotó de forma conservadora por el crédito limitado de la cuenta académica utilizada, lo que no necesariamente refleja el dimensionamiento óptimo para un entorno productivo con más usuarios.

11.3 Propuestas de mejora para una siguiente iteración

- Migrar la base de datos a Azure Database for PostgreSQL con alta disponibilidad, eliminando el punto único de fallo descrito.
- Incorporar TLS extremo a extremo (Application Gateway o Front Door) y autenticación con hash de contraseñas o un proveedor de identidad administrado.
- Configurar descubrimiento automático de instancias en Prometheus (azure_sd_config) para que el monitoreo cubra el autoescalado por completo.
- Incorporar un pipeline de integración/despliegue continuo que aplique Terraform y Ansible automáticamente ante cambios en el repositorio, reduciendo pasos manuales.

11.4 Uso de Inteligencia Artificial

Se utilizó Claude (Anthropic), a través de Claude Code, como herramienta de apoyo durante el desarrollo de este proyecto, por las siguientes razones: (1) apoyo en la redacción y estructuración de este Documento Técnico de Solución conforme al formato exigido por la evaluación; (2) revisión de la configuración de Terraform y Ansible ya escrita, lo que permitió detectar y corregir a tiempo una modificación no intencionada que sobredimensionaba las máquinas virtuales (de Standard_B1s/B2s a Standard_D2s_v7), evitando un gasto innecesario del crédito de Azure for Students; y (3) generación del diagrama de arquitectura de la sección 3 a partir de los recursos efectivamente definidos en el código Terraform del proyecto. Todo el código de infraestructura, la aplicación y las pruebas fueron definidos previamente como parte del desarrollo propio del proyecto; la herramienta de IA se usó como apoyo de redacción y revisión, no para generar la solución completa desde cero.

12. Referencias

International Organization for Standardization. (2015). ISO/IEC 27017:2015 — Information technology — Security techniques — Code of practice for information security controls based on ISO/IEC 27002 for cloud services. ISO. <https://www.iso.org/standard/43757.html>

International Organization for Standardization. (2019). ISO/IEC 27018:2019 — Information technology — Security techniques — Code of practice for protection of personally identifiable information (PII) in public clouds acting as PII processors. ISO. <https://www.iso.org/standard/76559.html>

Microsoft. (s.f.). Azure Well-Architected Framework. Microsoft Learn. <https://learn.microsoft.com/azure/well-architected/>

Microsoft. (s.f.). ¿Qué es Azure Load Balancer? Microsoft Learn. <https://learn.microsoft.com/azure/load-balancer/load-balancer-overview>

Microsoft. (s.f.). Virtual Machine Scale Sets. Microsoft Learn. <https://learn.microsoft.com/azure/virtual-machine-scale-sets/overview>

Microsoft. (s.f.). Descripción general del autoescalado en Azure. Microsoft Learn. <https://learn.microsoft.com/azure/azure-monitor/autoscale/autoscale-overview>

Microsoft. (s.f.). Seguridad de datos y cifrado en tránsito. Microsoft Learn. <https://learn.microsoft.com/azure/security/fundamentals/encryption-overview>

Prometheus Authors. (s.f.). Prometheus documentation. <https://prometheus.io/docs/introduction/overview/>

Grafana Labs. (s.f.). Grafana documentation. <https://grafana.com/docs/grafana/latest/>

HashiCorp. (s.f.). Terraform documentation. <https://developer.hashicorp.com/terraform/docs>

Red Hat. (s.f.). Ansible documentation. <https://docs.ansible.com/>